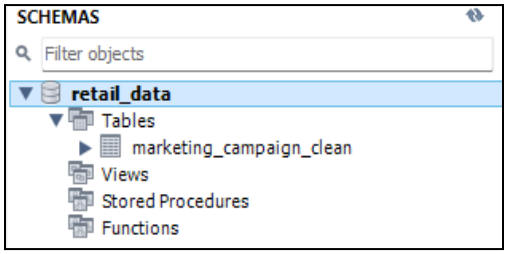
Marketing Campaign Data Analysis Report

1. Introduction

This report presents SQL-based data preprocessing and analysis performed on the marketing campaign dataset. It includes customer segmentation, campaign response analysis, product spending insights, and demographic trends.

2. Data Loading and Setup

A database named retail_data was created and used as the working schema. The marketing_campaign_clean table was imported using the SQL import wizard.



SQL Script:

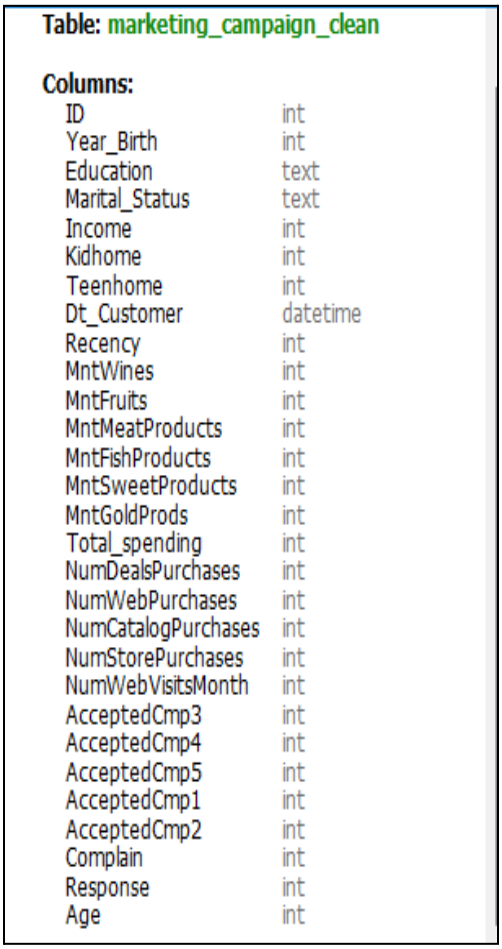
```
create database if not exists retail_data;
USE retail_data;
```

3. Data Cleaning

The Dt_Customer field was converted into a proper DATETIME format to ensure consistency across date values.

SQL Script:

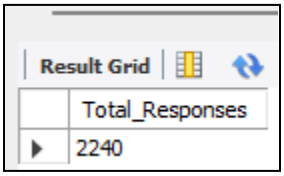
```
UPDATE marketing_campaign_clean
SET Dt_Customer =
    CASE
        WHEN Dt_Customer LIKE '%/%'
        THEN STR_TO_DATE(Dt_Customer, '%m/%d/%Y')
        WHEN Dt_Customer LIKE '%-%'
        AND Dt_Customer NOT LIKE '%:%'
        THEN STR_TO_DATE(Dt_Customer, '%d-%m-%Y')
        ELSE Dt_Customer
END;
ALTER TABLE marketing_campaign_clean
MODIFY Dt_Customer DATETIME;
```



4. Key Analyses

4.1 Total Customer Responses

Calculated total number of records in the dataset using COUNT(*).



SQL Script:

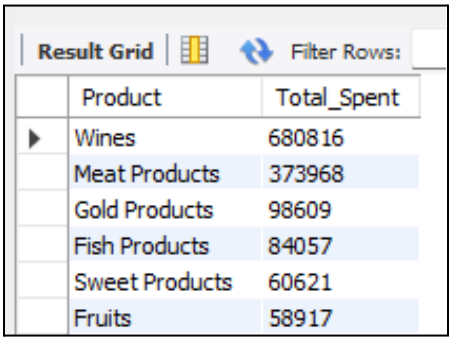
```
SELECT COUNT(*) AS Total_Responses
FROM marketing_campaign_clean
```

4.2 Product Spending Analysis

Aggregated spending across product categories (Wines, Fruits, Meat, Fish, Sweet, Gold) to identify top purchased products.

SQL Script:

```
SELECT 'Wines' AS Product, SUM(MntWines) AS Total_Spent FROM marketing_campaign_clean
UNION ALL
SELECT 'Fruits', SUM(MntFruits) FROM marketing_campaign_clean
UNION ALL
SELECT 'Meat Products', SUM(MntMeatProducts) FROM marketing_campaign_clean
UNION ALL
SELECT 'Fish Products', SUM(MntFishProducts) FROM marketing_campaign_clean
UNION ALL
SELECT 'Sweet Products', SUM(MntSweetProducts) FROM marketing_campaign_clean
UNION ALL
SELECT 'Gold Products', SUM(MntGoldProds) FROM marketing_campaign_clean
ORDER BY Total_Spent DESC;
```

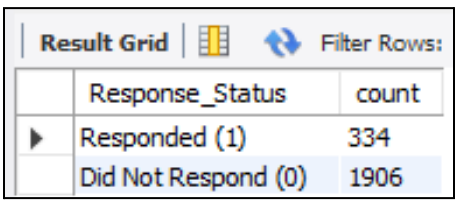


4.3 Campaign Response Distribution

Analyzed customer responses to the last campaign by grouping responses into 'Responded' and 'Not Responded'.

SQL Script:

```
SELECT
    CASE
        WHEN Response = 1 THEN 'Responded (1)'
```



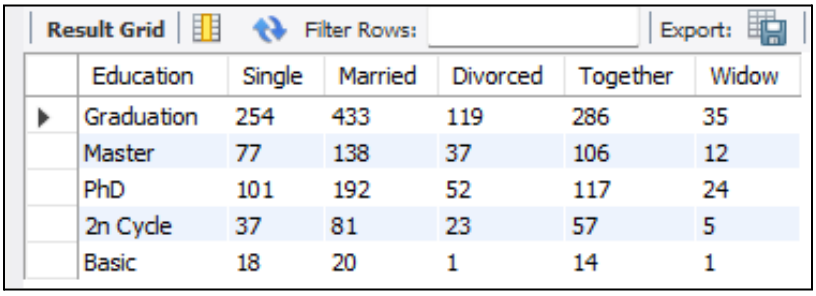
```
        ELSE 'Did Not Respond (0)'
        END AS Response_Status,
        COUNT(*) AS count
FROM marketing_campaign_clean
GROUP BY Response;
```

4.4 Customer Demographics

Examined distribution of customers based on education level and marital status.

SQL Script:

```
SELECT
        Education,
        SUM(CASE WHEN Marital_Status = 'Single' THEN 1 ELSE 0 END) AS Single,
        SUM(CASE WHEN Marital_Status = 'Married' THEN 1 ELSE 0 END) AS Married,
        SUM(CASE WHEN Marital_Status = 'Divorced' THEN 1 ELSE 0 END) AS Divorced,
        SUM(CASE WHEN Marital_Status = 'Together' THEN 1 ELSE 0 END) AS Together,
        SUM(CASE WHEN Marital_Status = 'Widow' THEN 1 ELSE 0 END) AS Widow
FROM marketing_campaign_clean
GROUP BY Education;
```


Result Grid | Filter Rows: | Export:

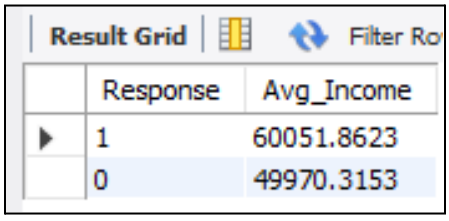
Education	Single	Married	Divorced	Together	Widow
Graduation	254	433	119	286	35
Master	77	138	37	106	12
PhD	101	192	52	117	24
2n Cycle	37	81	23	57	5
Basic	18	20	1	14	1

4.5 Income Analysis

Calculated average income of customers based on their campaign response behavior.

SQL Script:

```
SELECT
    Response,
    AVG(Income) AS Avg_Income
FROM marketing_campaign_clean
GROUP BY Response;
```


Result Grid | Filter Ro

Response	Avg_Income
1	60051.8623
0	49970.3153

4.6 Campaign Performance

Measured acceptance rates of multiple marketing campaigns.

SQL Script:

```
SELECT 'Campaign 1' AS Campaign, SUM(AcceptedCmp1) AS Accepted, COUNT(*) - SUM(AcceptedCmp1) AS Not_Accepted FROM marketing_campaign_clean
UNION ALL
SELECT 'Campaign 2', SUM(AcceptedCmp2), COUNT(*) - SUM(AcceptedCmp2) FROM marketing_campaign_clean
UNION ALL
SELECT 'Campaign 3', SUM(AcceptedCmp3), COUNT(*) - SUM(AcceptedCmp3) FROM marketing_campaign_clean
UNION ALL
SELECT 'Campaign 4', SUM(AcceptedCmp4), COUNT(*) - SUM(AcceptedCmp4) FROM marketing_campaign_clean
UNION ALL
SELECT 'Campaign 5', SUM(AcceptedCmp5), COUNT(*) - SUM(AcceptedCmp5) FROM marketing_campaign_clean;
```

Result Grid | Filter Rows:

Campaign	Accepted	Not_Accepted
Campaign 1	144	2096
Campaign 2	30	2210
Campaign 3	163	2077
Campaign 4	167	2073
Campaign 5	163	2077

4.7 Responses Performance

Distribution of customer responses to the last campaign (responded vs not responded).

SQL Script:

```
SELECT
    Response,
    COUNT(*) AS Total_Customers
FROM marketing_campaign_clean
GROUP BY Response
ORDER BY Total_Customers DESC;
```

Result Grid | Filter Rows:

Response	Total_Customers
0	1906
1	334

4.8 Average Children and Teenagers per household

Calculated the average number of children and teenagers in customers' households.

SQL Script:

```
SELECT
    AVG(Kidhome) AS Avg_Children,
    AVG(Teenhome) AS Avg_Teenagers
FROM marketing_campaign_clean;
```

Result Grid | Filter Rows:

Avg_Children	Avg_Teenagers
0.4442	0.5063

4.9 Age Analysis

Created an Age column and grouped customers into age segments to analyze behavior patterns using the following grouping:

WHEN Age BETWEEN 18 AND 25 THEN '18-25'

WHEN Age BETWEEN 26 AND 35 THEN '26-35'

WHEN Age BETWEEN 36 AND 45 THEN '36-45'

WHEN Age BETWEEN 46 AND 55 THEN '46-55'

ELSE '56+

SQL Script:

```
ALTER TABLE marketing_campaign_clean
ADD Age INT;
UPDATE marketing_campaign_clean
SET Age = YEAR(CURDATE()) - Year_Birth;
SELECT ID, Year_Birth, Age
FROM marketing_campaign_clean LIMIT 5;
```

Result Grid | Filter Row

ID	Year_Birth	Age
7734	1993	33
4369	1957	69
433	1958	68
7660	1973	53
92	1988	38

SQL Script:

```
SELECT
   Age_Group,
   AVG(NumWebVisitsMonth) AS avg_monthly_visits
FROM (
   SELECT
      NumWebVisitsMonth,
      CASE
         WHEN Age BETWEEN 18 AND 25 THEN '18-25'
         WHEN Age BETWEEN 26 AND 35 THEN '26-35'
         WHEN Age BETWEEN 36 AND 45 THEN '36-45'
         WHEN Age BETWEEN 46 AND 55 THEN '46-55'
         ELSE '56+'
      END AS Age_Group
   FROM marketing_campaign_clean ) AS grouped_data
GROUP BY Age_Group
ORDER BY Age_Group;
```

Result Grid | Filter Rows:

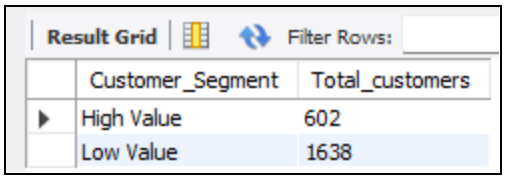
Age_Group	avg_monthly_visits
26-35	4.4419
36-45	5.5614
46-55	5.7094
56+	5.0373

4.10 Customer Segmentation

Customers were classified into High Value and Low Value groups based on total spending (threshold: $1000).

SQL Script:

```
SELECT
    CASE
        WHEN (Total_spending) > 1000
        THEN 'High Value'
        ELSE 'Low Value'
    END AS Customer_Segment,
    COUNT(*) AS Total_customers
FROM marketing_campaign_clean
GROUP BY Customer_Segment;
```



Result Grid | Filter Rows:

Customer_Segment	Total_customers
High Value	602
Low Value	1638

5. Conclusion

The analysis provides valuable insights into customer behavior, campaign effectiveness, and product preferences. These findings can support targeted marketing strategies and customer segmentation decisions.